

Introdução a React.js

Biblioteca JavaScript para construção de interfaces



Leonardo Ronchini

Introdução a React.js

O curso será dividido em 5 aulas, abordando os seguintes temas:

1. Revisão e introdução ao React;
2. Conhecendo o ambiente;
3. Componentes e Props;
4. Hooks e States;
5. Finalizando o projeto e extras.



O mercado de React

- **React** é usado por **44,7%** dos devs do mundo
- Mais de **3.000 vagas** abertas no Brasil
- Salário mensal bruto no Brasil (estimativa): **Júnior** R$ 2,5 a 5,5 mil, **Pleno** R$ 5,5 a 10 mil, **Sênior** R$ 10 a 18 mil;
- Remoto para empresas dos EUA: **pleno US$ 54 mil/ano** e **sênior US$ 66 mil/ano**;





Por que programar na era da IA?

- A IA barateou o ato de **escrever** código, mas não o de **entender** código;
- Jovens de 22 a 25 anos em desenvolvimento de software tiveram queda de **13%** no emprego em relação a outras áreas;
- A contratação de recém-formados caiu para cerca de **metade** do nível pré-pandemia;
- Profissionais **experientes** mantiveram o emprego estável ou em crescimento;
- Quem domina a **base** se diferencia em um mercado mais concorrido;



Entrevistas técnicas e LeetCode

- **LeetCode**, algoritmos e estruturas de dados continuam nas entrevistas das grandes empresas;
- Empresas já testam entrevistas com **IA liberada**: Google e Meta;
- O foco muda: **ler, testar e depurar** o código que a IA gerou;
- Quem depende da IA sem **entender** o código recebe feedback negativo;
- Você precisa **explicar** cada decisão, com ou sem IA;



Avaliar o código gerado pela IA

- **84%** dos devs usam IA, mas só **33%** confiam na precisão;
- Para **66%**, o maior problema é a solução "quase certa, mas não totalmente";
- **45%** do código gerado por IA trouxe falhas de segurança;
- Devs experientes ficaram **19% mais lentos** com IA, achando que estavam mais rápidos;
- Busque **orientação**: os mais experientes são os mais céticos com a IA;

O que é Front-end?

- **Front-end** é a parte do sistema que o usuário vê e usa;
- Cuida da **interface**, da **interação** e da experiência de uso;
- Conversa com o **back-end** (servidor e banco de dados) por meio de **APIs**;
- É construído com **HTML**, **CSS** e **JavaScript**;
- Exemplos: telas de um banco, formulários, menus e carrinho de compras;



Recapitulando o Front-end

- Toda página web é construída em **três camadas**;
- **HTML**: estrutura e conteúdo;
- **CSS**: aparência e layout;
- **JavaScript**: comportamento e interatividade;
- O React é construído **em cima** dessas três bases;

Vamos revisar cada uma, começando pelo **HTML**!



O que é HTML?

- **HTML** é a linguagem de **marcação** usada para estruturar páginas;
- Define a **organização** do conteúdo (títulos, parágrafos, imagens, links, listas);

- Usa **Tags** para representar elementos;
- Forma a base de qualquer página Web;
- É interpretado diretamente pelo navegador;



O que é CSS?

- **CSS** é a linguagem de **estilo** usada para personalizar páginas web;
- Define cores, fontes, tamanhos, espaçamentos e posicionamentos;



- Separa o **conteúdo** (HTML) da **aparência** (CSS);
- Permite aplicar **estilos** em elementos por classes e IDs;
- É essencial para criar **layouts responsivos** e modernos.

O que é JavaScript?

- **JavaScript** é a linguagem de **programação** da web;
- Permite adicionar **interatividade** às páginas (cliques, animações, validações);

- Manipula o **DOM**(Document Object Model) para alterar conteúdo em **tempo real**;
- Trabalha com variáveis, funções, objetos e eventos;
- É a base para bibliotecas e frameworks como React, Angular e Vue.







Introdução a React.js

Introdução da seção

O que é React?

- React é uma **biblioteca JavaScript** para desenvolvimento de aplicações front-end;
- A categoria delas é **SPA** (Single Page Application);
- Podemos criar uma aplicação com React, ou inserir em um projeto já em andamento;
- A sua arquitetura é baseada em **componentes**;
- É mantido pelo **Facebook/Meta**;





Por que usar React?

- **Componentes**: a interface é dividida em peças reutilizáveis;
- **Declarativo**: você descreve como a tela deve ficar e o React atualiza;
- **Desempenho**: atualiza só o que mudou na tela;
- **Ecossistema** gigante de bibliotecas, ferramentas e comunidade;
- **Mercado**: base para **React Native** (mobile) e **Next.js** (web);



Pré requisitos para rodar React

- Para iniciar uma aplicação React da maneira convencional precisaremos de **Node.js**;
- Através do gerenciador de pacotes **npm**, é possível iniciar projetos;
- Vamos ver como instalar o Node!





Node.js

- **Node.js** é um programa que permite rodar JavaScript fora do navegador.
- Ele é como o “**motor**” que executa o código.
- Com o Node.js, conseguimos:
  - Usar o **NPM** (gerenciador de pacotes).
  - Criar **servidores** ou rodar ferramentas no computador.
  - Executar projetos modernos como o **React**.



Node.js → npm → instalar pacotes → Vite → cria projeto React

NPM

- **NPM** (**Node Package Manager**) é o gerenciador de pacotes padrão do Node.js
- É usado para instalar **bibliotecas** (pacotes) JavaScript no seu projeto.
- Com o npm, você pode:
  - Instalar pacotes criados por outros desenvolvedores criaram
  - Gerenciar as dependências do projeto (bibliotecas que seu código precisa para rodar).
  - Automatizar tarefas usando scripts (definidos no package.json), como iniciar o servidor de desenvolvimento ou fazer o build para produção.



O que é o Vite?

O **Vite** é uma ferramenta moderna para criar e rodar projetos em **React** (ou outros frameworks como Vue, Svelte, etc.).
Ele substitui ferramentas mais antigas como o Create React App.
O que ele faz de especial:



- Cria a estrutura inicial do projeto (pasta, arquivos, configuração).
- Usa um **servidor de desenvolvimento muito rápido** para rodar a aplicação.
- Faz o **build** (empacotamento) do projeto de forma otimizada para produção.



Resumindo: o Vite é o "motor" que ajuda a gente a começar e rodar um projeto em React sem precisar configurar tudo do zero.

A importância do terminal

O que é o terminal

- Uma janela onde conversamos com o computador escrevendo comandos, em vez de clicar com o mouse
- Tudo o que fazemos clicando (abrir pasta, criar arquivo, instalar programa) também pode ser feito escrevendo
- Foi assim que os computadores foram usados por décadas, antes de existirem janelas e mouse
- Você escreve uma ordem, aperta Enter, o computador executa e responde na tela
- Nomes que você vai ouvir: terminal, prompt de comando, cmd, linha de comando (são variações da mesma ideia)





A importância do terminal

Por que programadores usam o terminal

- Muitas ferramentas de programação não têm botões nem janelas, só funcionam por comando (Node, npm, Vite, Git)
- É mais rápido (uma linha faz o que levaria vários cliques)
- Dá para repetir (o mesmo comando funciona igual em qualquer computador)
- Dá para automatizar (comandos podem ser guardados em um arquivo e executados sempre que precisar)
- É assim que se trabalha em empresas e em servidores (que quase nunca têm tela gráfica)





A importância do terminal

Como é um comando

- Todo comando tem a mesma estrutura: *programa o-que-fazer opções*
- Exemplo: npm install (programa npm, ação install: instala as dependências)
- O terminal sempre trabalha dentro de uma pasta (a pasta atual aparece antes do cursor, por exemplo C:\Users\Leo\react>)
- Depois de Enter, o terminal mostra o resultado ou uma mensagem de erro (ler a mensagem ajuda a descobrir o problema)





A importância do terminal

Alguns comandos

- cd (pasta) (entra na pasta; cd .. volta uma pasta)
- dir no Windows ou ls no Mac e Linux (lista o que tem na pasta atual)
- mkdir (nome) (cria uma pasta nova)
- cls no Windows ou clear no Mac e Linux (limpa a tela)
- node -v e npm -v (mostram a versão do Node e do npm, e confirmam que estão instalados)
- Ctrl + C (interrompe o que estiver rodando)





Criando o projeto em React

- **Abrir o terminal dentro da pasta onde o projeto vai ficar** (crie uma pasta, por exemplo react; no Explorador de Arquivos clique na barra de endereço, digite cmd e Enter.
- **node -v e npm -v** (confere se o Node e o npm estão instalados;
- **npm config set strict-ssl false** (desativa a verificação SSL, necessária para os computadores da PUC)
- **npm create vite@latest** (cria o projeto; o terminal faz algumas perguntas, escolha com as setas ↑ ↓ e confirme com Enter)



- Project name: digite aula2
- Select a framework: escolha React
- Select a variant: escolha JavaScript
- Which linter to use? escolha Eslint
- Install with npm and start now? escolha Yes



Para executar o projeto React

- **cd aula2** (entra na pasta do projeto)
- **npm install ou npm i** (instala as dependências)
- **code .** (abre a pasta do projeto no VS Code)
- **npm run dev** (executa a aplicação que está em React)

- **A aplicação já estará rodando.** Abra no navegador o endereço que aparecer, normalmente http://localhost:5173)

- Ctrl + C para encerrar a aplicação se necessário.





Por que não existe uma IDE “oficial” para React?

React não é uma linguagem, é uma biblioteca.

Java tem IDEs específicas (Eclipse, IntelliJ). C# tem o Visual Studio.

Ele roda em cima do JavaScript, que por sua vez pode ser escrito em qualquer editor ou IDE



React é multiplataforma e aberto, A ideia é funcionar em qualquer lugar, sem prender o desenvolvedor a uma ferramenta específica..

No mundo web, novas ferramentas aparecem o tempo todo (Webpack, Vite, Next.js, etc).



Se houvesse uma IDE oficial fixa, ela poderia se tornar ultrapassada rápido.

Por isso, a comunidade prefere usar editores genéricos como VS Code, que se adaptam com extensões.

E por que não tem interface gráfica para criar projeto?

Terminal é universal

Funciona no Windows, Linux, Mac, em qualquer lugar.

Muito mais fácil ensinar 3 comandos (npm create, npm install, npm run dev) do que manter 10 programas gráficos diferentes.



Automatização e padronização

Times de desenvolvimento precisam de processos repetíveis.

Com o terminal, todo mundo roda os mesmos comandos em qualquer máquina.



Uma interface gráfica poderia mudar de versão para versão, mas o comando no terminal é sempre o mesmo.

Como os devs trabalham então?

O que os desenvolvedores fazem é usar uma IDE genérica (como o **VS Code**), mas adicionam:

Extensões para React e JavaScript (ex: React Developer Tools, ESLint, Prettier).

O terminal integrado (no próprio VS Code você já consegue rodar os comandos npm).

Assim você tem o melhor dos dois mundos:

Uma interface amigável para escrever código.

O terminal para instalar, rodar e controlar o projeto.





Estrutura base do React

- Há algumas pastas e arquivos muito importantes;
- **node_modules**: dependências do projeto;
- **public**: assets e arquivos estáticos;
- **src**: onde escrevemos o código da aplicação;
- **src/index.js**: arquivo de inicialização da aplicação;
- **src/App.js**: componente principal inicial (pode ser modificado);





Fundamentos do React.js

Introdução da seção



O que são componentes?

- **Componentes** são como “peças de LEGO” que juntas formam a interface.
- Cada componente representa uma parte da interface (botão, menu, card, etc.).
- Componentes podem ser acompanhados e **propriedades** e **estados**.
- É a base do **React**.



Por que usar componentes?

- **Reutilização** → não precisamos repetir o mesmo código várias vezes.
- **Organização** → cada parte da tela fica em um arquivo separado.
- **Facilidade de manutenção** → mudar um componente reflete em todos os lugares.



- Em resumo, componente é apenas uma função que retorna **JSX**



3 regras de um componente

- O **nome** começa com letra **maiúscula**.
- Usamos como uma **tag**: <Filme />.
- Um componente pode usar outros componentes.





O que é o JSX

- **JSX** é uma forma de escrever o **HTML** dentro do **JavaScript**. O navegador não entende, então o **Vite** converte por baixo. É uma extensão da sintaxe JavaScript.
- Permite escrever código que parece HTML dentro do JavaScript.
- **JSX** é transformado em funções React pelo compilador (Vite/Babel).
- Ou seja, **JSX** é o **HTML** do **React**, o código interno das funções de componentes, após o return;
  - Podemos escrever **JavaScript** dentro do **JSX**;
  - Há algumas diferenças do HTML, ex: **class** = **className**;

Criando componentes

- Os componentes ficam dentro de uma pasta chamada **components**, que criamos em src;
- Nomeados em CamelCase: **FirstComponent.jsx**;
- A utilização da extensão **.jsx** facilita a formatação para os editores;
- Dentro do componente precisamos **criar e exportar uma função**, que é a lógica dele;
- Vamos ver na prática!



Importando componentes

- Para utilizar e reutilizar um componente é necessário o processo de **importação**;
- A sintaxe é: **import X from ‘./components/X.jsx’** onde X é o nome do componente;
- Para inserir o componente dentro de outro vamos utilizar a sintaxe de tag do HTML com o nome do componente: **<FirstComponent />**
- Vamos ver na prática!





Template Expressions

- **Template Expression** é o recurso que permite a execução de JS no JSX;
- Podemos também inserir variáveis;
- A sintaxe é: **{ 2 + 2 }**
- Tudo que vai entre as chaves é entendido e executado como JavaScript;
- Vamos ver na prática!





Props (Parâmetros do Componente)

- As **props** (abreviação de properties, ou propriedades) são os dados que um **componente pai** envia para um **componente filho** no ReacT
- Elas funcionam como os **argumentos** de uma função tradicional. Servem para deixar os componentes **dinâmicos** e **reutilizáveis**.
- Fluxo **unidirecional**: Os dados viajam apenas em um sentido - do pai para o filho.
- Somente leitura (**read-only**): O componente que recebe a prop não pode modificá-la. Quem cria ou altera a prop é sempre o componente pai.





O que são hooks?

- Uma variável comum muda o valor na memória, mas não atualiza a tela;
- **Hooks** são funções do React que dão memória e **comportamento** ao **componente**;
- Com eles o componente guarda um dado e redesenha a tela quando esse dado muda;
- Exemplo: const [contador, setContador] = useState(0);
- contador é o valor atual, setContador troca o valor, 0 é o valor inicial;
- Alterou pelo setContador → o React **atualiza a tela sozinho**;



Regras e tipos de hooks

- Todo hook começa com a palavra **use** — é assim que você reconhece um;
- Precisa ser **importado**: import { useState } from 'react';
- Só funciona **dentro de um componente**, e sempre no **topo** da função;
- Nunca chame um hook dentro de **if**, **for** ou função aninhada;
- Os mais usados: **useState** (guardar dados) e **useEffect** (rodar algo quando o componente carrega ou muda);
- Você também pode criar os seus: são os **custom hooks**;





hook: useState

- O **useState** é o hook mais popular do react. Seu uso em projetos é crucial.
- A **estrutura** é sempre a mesma: const [dado, setDado] = useState(inicial);
- **Convenção**: o setter é o nome do dado com **set** na frente — setContador;
- Você **lê** o valor pelo nome, mas **nunca** escreve nele: contador = 5 não funciona;
- Toda mudança passa pelo **setter** - é ele que manda o React redesenhar a tela;
- O valor inicial aceita **número**, **texto**, **lista** ou **objeto**;
- Um componente pode ter **vários useState**, um para cada dado;





State e props

- **Props:** Dados que vêm *de fora* (do componente pai); são estáticos para quem os recebe.
- **State (Estado):** Dados gerenciados *internamente* pelo próprio componente e que podem mudar ao longo do tempo
- Ou seja: **Props** são o que o componente **recebe**; **state** é o que ele **guarda**;
- Props vêm **do pai**, e o componente **não pode alterá-las**;
- State nasce **dentro** do componente, que o altera pelo **setter**;
- <Aluno nome="Maria" /> é props; useState('Maria') é state;
- Os dois, ao mudar, fazem o React **redesenhar a tela**;
- Na dúvida: veio de fora é **props**, muda por ação daqui é **state**;





O que é uma API?

- **API** é a porta que um sistema abre para outro pedir dados;
- Você manda um **pedido** para uma **URL** e recebe uma **resposta**;
- A resposta quase sempre vem em **JSON** - pares de **chave** e **valor**;
- É como um restaurante: você pede o que está no cardápio ao garçom e não entra na cozinha;
- **Consumir** uma **API** é fazer esse pedido e usar os dados na sua aplicação;
- O garçom é o **frontend**, o cardápio é a **API**, a cozinha é o **backend**.
- Para esse projeto, não criaremos o backend e sim usaremos um backend já pronto, ou seja, uma **API pública**.





Consumindo uma API em React

- **fetch()** faz o pedido e devolve uma **Promise** — a resposta **não chega na hora**;
- **useEffect** dispara o pedido quando o componente **nasce**;
- **useState** guarda o que voltou e faz a tela se redesenhar;
- O [] no fim do **useEffect** significa **rode uma vez só**;
- Toda **API pública** tem uma **chave**, essa chave normalmente é guardada dentro do arquivo **.env** que não é commitado para o GIT (**nunca fica pública**)





API do OMDB (filmes)

- **OMDb** (Open Movie Database) devolve dados de **filmes e séries**;
- Toda chamada precisa da sua **apikey** na URL;
- Um filme pelo id do IMDb: ?i=tt3896198
- Uma lista por nome: ?s=batman
- Se der erro, vem "Response": "False"

Exemplo de resposta da API:

{
"Title": "Guardians of the Galaxy: Vol. 2",
"Year": "2017",
"Genre": "Action, Adventure, Comedy",
"Director": "James Gunn",
"imdbRating": "7.6",
"Poster": "https://m.media-amazon.com/..."
}



Quem não é visto não é lembrado

Introdução da seção



GitHub

O que é:

Plataforma para hospedar código. Usa **Git** (sistema de controle de versão).

GIT **!=** GitHub/GitLab/BitBucket/AzureDevops



Permite:

- Guardar o histórico dos seus **projetos**.
- Trabalhar em **equipe** (pull requests, branches).
- Mostrar seu código para empresas e **recrutadores**.



Por que GitHub é importante?

- Funciona como um **portfólio técnico**.
- Centralização e gerenciamento de **projetos pessoais**
- Mostra o que você já fez, já sabe e o que você pratica.
- Expõe seu conhecimento a **recrutadores**.



Clonar repositório

O processo de clone de um repositório permite você trazer um projeto público do github para seu computador.

Comandos:

- Acessar o repositório - **https://github.com/leoronchini/-seminario-react-puc**

- Clicar no botão Code e copiar o link
- Pode ser configurado via HTTPS, SSH ou GitHub CLI.
- Pode ser configurado também via github desktop ou apenas baixar o ZIP do projeto.



Git Clone

Comandos:

Abrir um novo terminal e verificar se o GIT está disponível. Se estiver, rodar o comando de clone.

Com seu git configurado, rode os comandos a seguir

git --version

git clone <colar o link copiado do github>



Criar um repositório no Github pessoal

- Acessar o **GitHub** - https://github.com/
- Logar em sua conta
- Clique no botão + no canto direito superior e selecione **New Repository**
- Em **Repository Name** escreva **projeto-react-puc**
- Na **descrição**:

  Projeto desenvolvido em aula durante o seminário de Introdução ao React.Js no segundo período do curso de Ciência da Computação - PUC Minas Poços de Caldas

- Clique em **Create Repository**





Antes de publicar o projeto, o .gitignore

- O **.gitignore** lista o que o git deve **ignorar** ao subir o projeto;
- Sem ele, o **node_modules** inteiro vai junto - **dezenas de milhares** de arquivos;
- Nada se perde: o package.json lista as dependências e o **npm install** refaz a pasta;
- O **Vite já cria** esse arquivo; se a pasta veio de outro lugar, crie na raiz do projeto;
- No **.env** ficam chaves e senhas - subir esse arquivo **expõe** sua **API key**;
- Confira com **git status**: deve aparecer algumas dezenas de arquivos, nunca milhares;





Publicar o projeto no seu Github

Comandos:

git init (iniciar o git)

git add . (mandar todas alterações para stage)

git commit -m "primeiro commit" (escrever a mensagem do commit)

git branch -M main (selecionar a branch)

git remote add origin https://github.com/usuario/repositorio.git (conectar com seu repositorio)

git push -u origin main (fazer o push do commit)





Importante:

Após o primeiro push, a sequência será apenas esta:

git add ., git commit -m "..." e git push.

Possiveis problemas:

1. O push é recusado
O repositório foi criado com README ou .gitignore pelo GitHub, então o remoto tem um commit que você não tem.

git pull --rebase origin main
git push -u origin main

2. A pasta já era um repositório git
Pule o git init. Se o origin aponta para outro lugar:

git remote -v
git remote set-url origin https://github.com/usuario/repositorio.git

3. Ele pede usuário e senha
A senha é o **personal access token**, nunca a senha da conta.

git config --global credential.helper store

Configurar access token (caso necessário)

1. Acesse o github e vá para **Settings** (Configurações).
2. No menu lateral esquerdo, vá em **Developer settings** (Configurações do desenvolvedor).
3. Clique em **Personal access tokens → Tokens (classic)**.
4. Clique no botão verde **Generate new token (classic)**.
5. **Note:** Coloque um nome (Ex: Token Pokedex).
6. **Expiration:** Defina um prazo (7 dias ou 30)
7. **Select scopes:** Marque a caixa **repo** (isso permite que o token leia e escreva nos repositórios).
8. Clique no botão verde **Generate token** na parte inferior.





⚠ **MUITO IMPORTANTE:** O token gerado (uma sequência longa de letras e números) só será exibido **uma vez**. Copie e guarde o token.

Linkedin

O seu projeto agora está publicado no **Github** porém é muito importante ser visto pelas **empresas** e recrutadores.

A forma mais fácil de fazer isso criando uma publicação no **Linkedin**.



Por quê Linkedin?

- Plataforma com mais usuários e empresas ativas;
- Possibilidade de demonstrar conhecimentos e habilidades e fazer **networking**.
- A melhor de uma empresa saber do seu conhecimento é através de uma **publicação**.

Vamos publicar nosso projeto!

Publicação do projeto no Linkedin

Clique em começar uma publicação.Texto sugerido:

Hoje finalizei meu primeiro projeto em React: uma Biblioteca de filmes que consome dados do OMBD api.

O projeto foi desenvolvido durante o seminário de Introdução ao React.js do curso de Ciência da Computação na PUC Minas Poços de Caldas, ministrado por @Leonardo Ronchini

Aprendi e apliquei conceitos de componentes, hooks, props e consumo de APIs.

Código disponível no GitHub: <Link do seu repositório>

#react #cienciadacomputacao #desenvolvimentoweb

Inserir uma imagem do projeto